Architecture A and Architecture B

OCR → LLM invoice extraction: how the two pipelines are built, how they were benchmarked, and how they compare on 600 invoices

Project: Latency and Accuracy Optimization for OCR to LLM Document Processing Pipelines · 20 October 2026 · Benchmark: `outputs/2026-10-18_23-52-41_full_benchmark`

5.7×	**0.941**	**5.7×**	**1.9×**
throughput of B over A 53.1 → 301.7 invoices/min	F1 of A and of B same extraction in both	lower cost per invoice in B €0.0110 → €0.0019 per 100	more GPU memory in B 1067 → 1996 MB

Contents 1. Summary · 2. Task and shared components · 3. Architecture A · 4. Architecture B · 5. Benchmark method · 6. Results · 7. Is OCR still needed? · 8. Improvement options · 9. Limits · Appendix: reproducing the benchmark

1. Summary

Both architectures run the same models on the same 600 invoices. B finishes them 5.7× faster (119 s instead of 678 s) with the same accuracy (F1 0.941).

- **Where the speed comes from:** three OCR processes work in parallel (2.8× on their own), the GPU answers 8 invoices per call (2.1× on their own), and OCR and GPU run at the same time. B's total time is set by whichever of its two stages is slower; on this laptop that is OCR.
- **What it costs:** 1.9× the GPU memory (1996 vs 1067 MB) and 1.4× the RAM, because B keeps two model copies and three worker processes in memory. Results also arrive in batches: B's first result comes after 4.4 s, A's after 0.6 s.
- **Quantization** on this 248 M-parameter model saves memory, not time: 4-bit uses a third less GPU memory than fp32 at the same speed and accuracy. fp16 is the best option on this GPU.
- **Cache:** the same 600 invoices submitted again return in 0.59 s.
- **Latency with invoices arriving one at a time:** B as benchmarked waits for 8 invoices, so at light load it is slower than A (median 7.9 s vs 1.7 s at 30 per minute). With dynamic batching B answers in 0.95–1.8 s at every load, while A's queue grows without limit above about 53 invoices per minute.
- **OCR engines:** Tesseract 0.57 s per invoice on the CPU, F1 0.941; EasyOCR 4.16 s on the GPU with 3883 MB of GPU memory, F1 0.934.
- **Vision-language model:** Qwen2-VL-2B (4-bit) reading the image directly is more accurate – F1 0.966 against B's 0.941 – but takes 3.9 s per invoice against B's 0.20 s. Sending only the 45 invoices B flags for review to the VLM would give as many fully correct invoices as the VLM alone (541 of 600) for about 0.29 s of extra GPU time per invoice (estimate).

2. The task and the shared components

2.1 The task

Input: one invoice image (PNG, JPG or WebP). Output: three fields as JSON. Both architectures solve exactly this task with exactly the same components and settings; they differ only in how the work is scheduled.

```
{"invoice_number": "INV-8289-16", "date": "April 19, 2024", "total": "$45,130"}
```

2.2 Components used by both architectures

Component	What it does	Code	Key settings
OCR	Tesseract 5 returns every word with its box and confidence; the text is rebuilt row by row so a label and its value stay on one line	`pipeline/ocr.py`	grayscale, EXIF-rotated; `OCR_LAYOUT="rows"`
Line filter	MiniLM embeds each text line and keeps the 6 lines most similar to each field (plus the line after each), so the prompt fits Flan-T5's 512 tokens	`pipeline/embeddings.py`	all-MiniLM-L6-v2; skipped when the text has ≤ 18 lines
LLM	Flan-T5-base (248 M parameters) answers one question per field	`pipeline/llm.py`	greedy decoding; at most 12 (FAST) or 24 (FULL) new tokens
Normalisation	One canonical form for invoice numbers, dates and amounts, used for the output and for scoring	`pipeline/normalize.py`	day/month swap tolerated; separator rules for , and .
Total check	If amounts on the total lines satisfy subtotal + tax = total, that total replaces the model's answer	`pipeline/validate.py`	`RECONCILE_TOTALS = True`
Review flag	Marks an invoice for a person: missing or unreadable field, total not on a total line, OCR confidence below 80	`pipeline/validate.py`	reasons stored per invoice
Scorer	TP / FP / FN per field, F1 per field and overall	`evaluation/scorer.py`	fuzzy match ≥ 90 for invoice numbers

2.3 Prompt format

Each field is asked as a separate reading-comprehension question, the format Flan-T5 was instruction-tuned on. Asking for JSON does not work with T5: its vocabulary has no { or } token.

```
<selected invoice lines>

Question: What is the total amount due?
Answer:
```

2.4 Model precisions

The LLM can be loaded in four precisions. A always uses fp32; B uses 4-bit on its FAST route and fp32 on its FULL route. The other precisions are benchmarked as variants of B (section 6.8).

Precision	How it is loaded	Weight memory
fp32	plain PyTorch	~990 MB
fp16	`dtype=torch.float16`	~500 MB
8-bit	bitsandbytes LLM.int8 (`load_in_8bit=True`)	~300 MB
4-bit NF4	bitsandbytes NF4 with double quantization (`load_in_4bit=True, bnb_4bit_quant_type="nf4"`)	~200 MB

3. Architecture A – the sequential baseline

A processes one invoice at a time in one process. Each step waits for the previous one, and the next invoice starts only when the current one is finished.

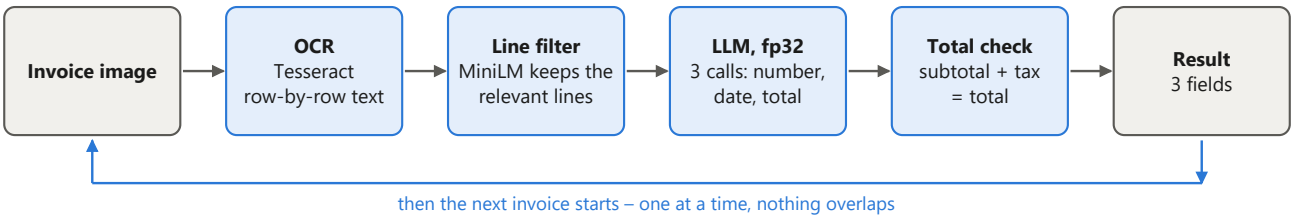


Figure 1 – Architecture A (code: `pipeline/arch_a.py`).

Step, per invoice	Where it runs	Mean time (benchmark)
OCR	CPU, main process	0.588 s
Line filter (MiniLM, one encode call)	GPU	0.017 s
LLM: three generate calls, fp32	GPU	0.514 s
Normalisation, total check, review flag	CPU	included above
Total per invoice		**1.119 s → 53.1 invoices/min**

- While OCR runs, the GPU is idle – about 53 % of the time. While the GPU runs, 11 of the 12 CPU threads are idle.
- Latency equals compute time: a single invoice is answered in about 1.1 s, and a stream of invoices is served at the same rate.
- The model and the GPU kernels are warmed up with a dummy invoice before timing starts, so the first real invoice does not pay CUDA initialisation. B gets the same warm-up.

4. Architecture B – the optimised pipeline

B receives all invoices at once. Already-seen invoices are answered from the cache; the rest flow through a pipeline in which OCR and the GPU work at the same time on different invoices.

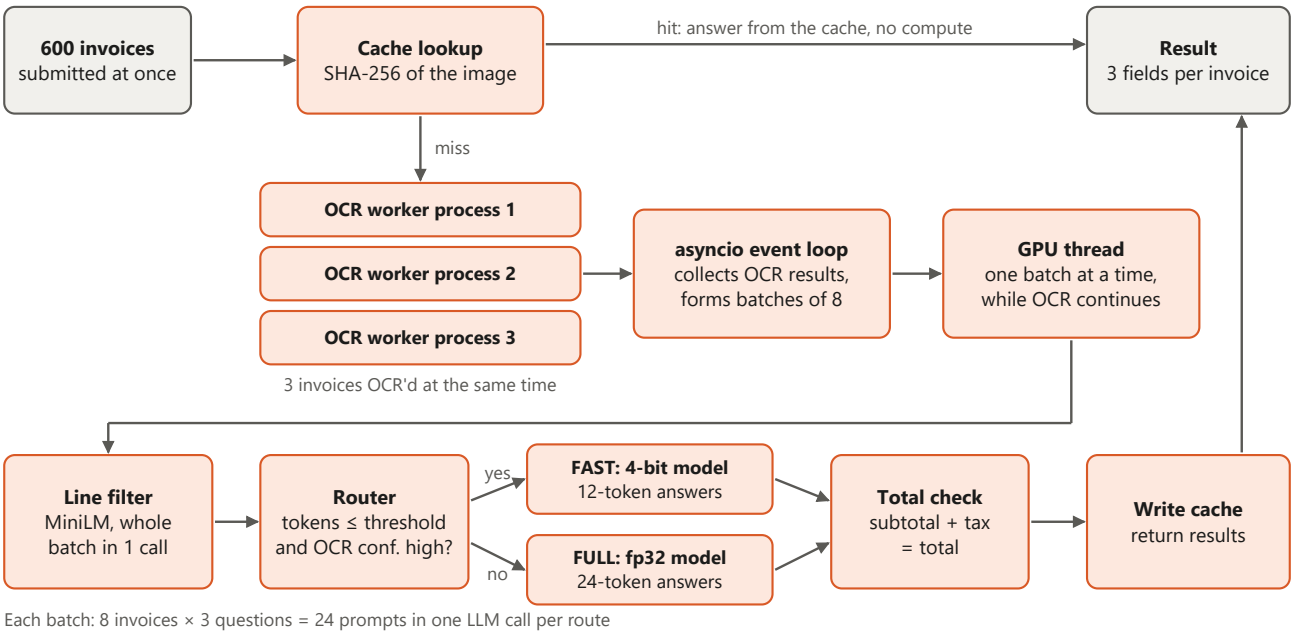


Figure 2 – Architecture B (code: `pipeline/arch_b.py`).

4.1 The parts and why each exists

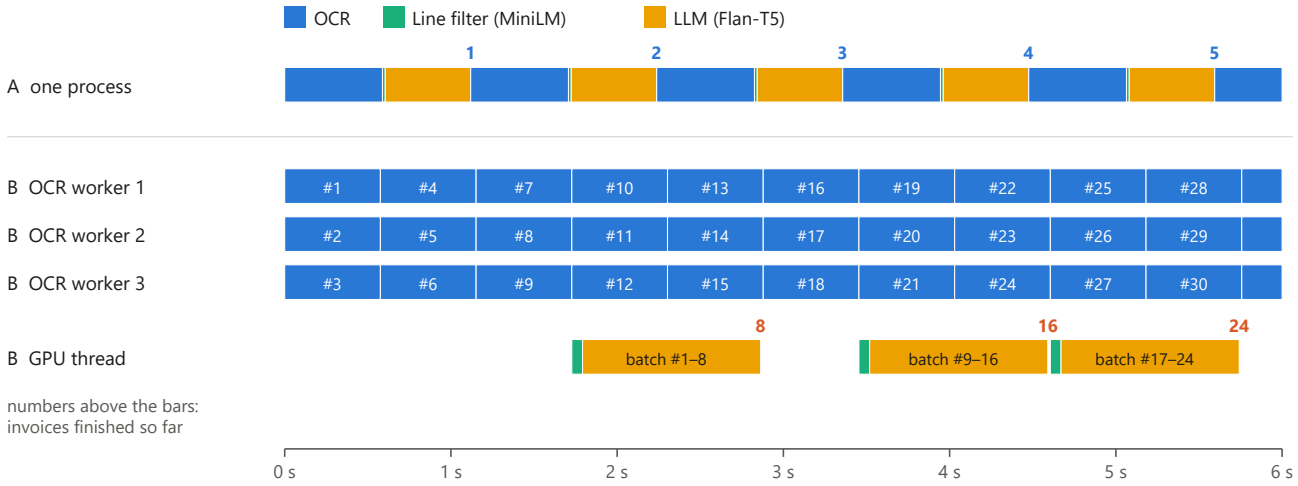
Part	What it does	Why
asyncio event loop	Submits OCR jobs with `run_in_executor` to the process pool and GPU batches to the GPU thread; collects OCR results in completion order	The loop never blocks, so while the GPU works on batch k the OCR workers already produce batch k + 1. asyncio alone would not help: OCR and inference are CPU- and GPU-bound, not I/O-bound
Process pool, 3 OCR workers	Each worker runs Tesseract on one invoice; workers import only Pillow and pytesseract (~60 MB each)	Processes, not threads, so the GIL does not serialise image decoding; `OMP_THREAD_LIMIT=1` keeps Tesseract's own threads from competing
One GPU thread	Runs line filter, routing and LLM for one batch at a time	Two concurrent generate calls on a 4 GB GPU would not be faster and could run out of memory; the GPU is kept busy by batching instead
Batches of 8	8 invoices × 3 questions = 24 prompts in one generate call, one MiniLM call per batch	One large call uses the GPU far better than 24 small ones
Router	Sends an invoice to FAST (4-bit, 12 tokens) when its OCR text has ≤ 300 tokens and mean OCR confidence ≥ 85, otherwise to FULL (fp32, 24 tokens)	Cheap model for easy invoices, full model for long or poorly scanned ones; thresholds are re-derived from the benchmark data (section 6.9)
Result cache	SQLite table keyed by SHA-256 of the image bytes plus the model and configuration	A renamed copy still hits; a changed image or configuration misses, so stale results cannot leak between experiments

4.2 Side by side

	Architecture A	Architecture B
Order of work	One invoice at a time, start to finish	All stages busy at once, on different invoices
OCR	Main process, one image at a time	3 worker processes, 3 images at a time
Line filter (MiniLM)	One call per invoice	One call per batch of 8 invoices
LLM calls	3 per invoice (one per field)	1 per batch and route: 8 invoices × 3 fields = 24 prompts
Model precision	fp32 only	Per invoice: 4-bit (FAST) or fp32 (FULL)
Repeated invoices	Processed again from scratch	Answered from the SQLite cache
Coordination	A plain loop	asyncio event loop, process pool, one GPU thread
Models resident on the GPU	Flan-T5 fp32, MiniLM	Flan-T5 fp32, Flan-T5 4-bit, MiniLM
Same in both	Tesseract, MiniLM, Flan-T5-base, prompts, normalisation, total check, review flag, scorer	

4.3 How the work is scheduled

Figure 3 draws the first six seconds of both architectures to scale, using the mean stage times measured in the benchmark. A runs its three stages one after another. In B, the three OCR workers run continuously; as soon as 8 invoices are read, the GPU processes them in one batch while OCR continues.



After 6 s: A has finished 5 invoices, B has finished 24. The GPU lane has gaps – it waits for OCR, so OCR sets B's pace.

Figure 3 – Idealised schedule of the first 6 s, drawn from the mean stage times of the benchmark (A: OCR 0.59 s, LLM 0.51 s per invoice; B: OCR 0.58 s per invoice and worker, GPU 1.14 s per batch of 8). Real OCR times vary by invoice, so real batches form less regularly.

4.4 Latency and service time in a batched system

- **Service time** = OCR time + the invoice's share of its batch's line-filter time + its share of its route's LLM time. It is the compute spent on one invoice and is directly comparable with A.
- **Latency** = time from submission to result. Because the benchmark submits all 600 invoices at once, B's latency includes queueing and grows with the invoice's position in the queue; its p95 latency is therefore close to its wall time. With invoices arriving one by one, B as benchmarked waits until 8 invoices are read before the GPU starts; section 6.11 measures this and a dynamic-batching variant that removes the wait.
- **Throughput** (invoices per minute of wall time) is the headline metric for B, because that is what parallelism and batching improve.

5. Benchmark method

5.1 Dataset

Subset	Invoices	Origin	Image formats
Real	1–15, 316–600 (300)	public invoice templates and scanned invoices, labelled by hand	WebP, PNG, JPG; 1131×1600 to 2481×3509 px
Synthetic	16–315 (300)	`generate_dataset.py`: rendered company name, number, date, line items, noise lines, total	PNG 800×600 or 800×1000

The synthetic half is clean and easy and serves as a sanity floor; the real half is where differences show, so accuracy is reported for both halves separately. The ground-truth total is the gross amount due.

5.2 Configurations

Each variant of B changes exactly one thing, so comparing it with B attributes the gain to that part.

Configuration	What changes	What it isolates
A	baseline	reference
B	all optimisations, cache cleared before the run	the optimised architecture, first pass
B-warm	B again immediately, cache full	steady-state benefit of the cache
B-nocache	no cache	cost of the cache lookup on a first pass
B-allfull	every invoice on fp32	B without the quantized model
B-allfast	every invoice on 4-bit	B with only the quantized model
B-fp16	every invoice on fp16	half precision instead of quantization
B-8bit	every invoice on LLM.int8	8-bit instead of 4-bit quantization
B-1worker	1 OCR worker	value of OCR parallelism
B-6workers	6 OCR workers	how far OCR parallelism scales
B-nobatch	batch size 1	value of GPU batching
B-tuned	routing thresholds from the data (525 tokens, confidence 70)	data-derived routing

5.3 Protocol

- All 600 invoices, **3 runs per configuration**; tables give mean ± standard deviation over the runs.
- Models are loaded and warmed up before the clock starts; loading time is reported separately and excluded from wall time.
- Models are loaded offline; the laptop is plugged in and idle; the result cache is cleared before every cold run.
- Runs disturbed by other programs were recognised by their OCR time per invoice, moved to `discarded/` with the reason in the folder name, and repeated.
- Generation is deterministic (greedy decoding), so accuracy can only differ between runs where timing affects the code path (8-bit's outlier handling is the one case observed).

5.4 Metrics

Metric	Definition
Wall time	seconds from submitting the first invoice to receiving the last result
Throughput	invoices ÷ wall time × 60
Service time	compute spent on one invoice (OCR + share of line filter + share of LLM)
Latency, p95	time from submission to result, 95th percentile over invoices
Peak RAM	maximum resident memory of the whole process tree (main process + OCR workers), sampled 4× per second
Pipeline GPU peak	PyTorch's peak allocated GPU memory during the run
Cost	wall time × €0.35 per GPU-hour (typical price of an entry-level cloud GPU), per 100 invoices
F1	2 · precision · recall ÷ (precision + recall), per field and over all fields
Straight-through rate	share of invoices accepted without a review flag

5.5 Hardware and software

Item	Version or model
GPU	NVIDIA GeForce RTX 3050 Laptop GPU, 4 GB VRAM
CPU and RAM	Intel x86-64 (Family 6 Model 154 Stepping 3), 12 logical cores; 15.7 GB RAM
Operating system	Windows 11 (build 22621)
Python, PyTorch, CUDA	Python 3.12.2, PyTorch 2.5.1+cu121, CUDA 12.1
Libraries	transformers 5.7.0, bitsandbytes 0.50.2, accelerate 1.15.0, sentence-transformers 6.1.0, pytesseract 0.3.13, rapidfuzz 3.14.6
OCR engine	Tesseract 5.4.0.20240606
Models	google/flan-t5-base; sentence-transformers/all-MiniLM-L6-v2

6. Results: A compared with B

6.1 Headline numbers

B is 5.7× faster and 5.7× cheaper per invoice; it pays for that with more memory.

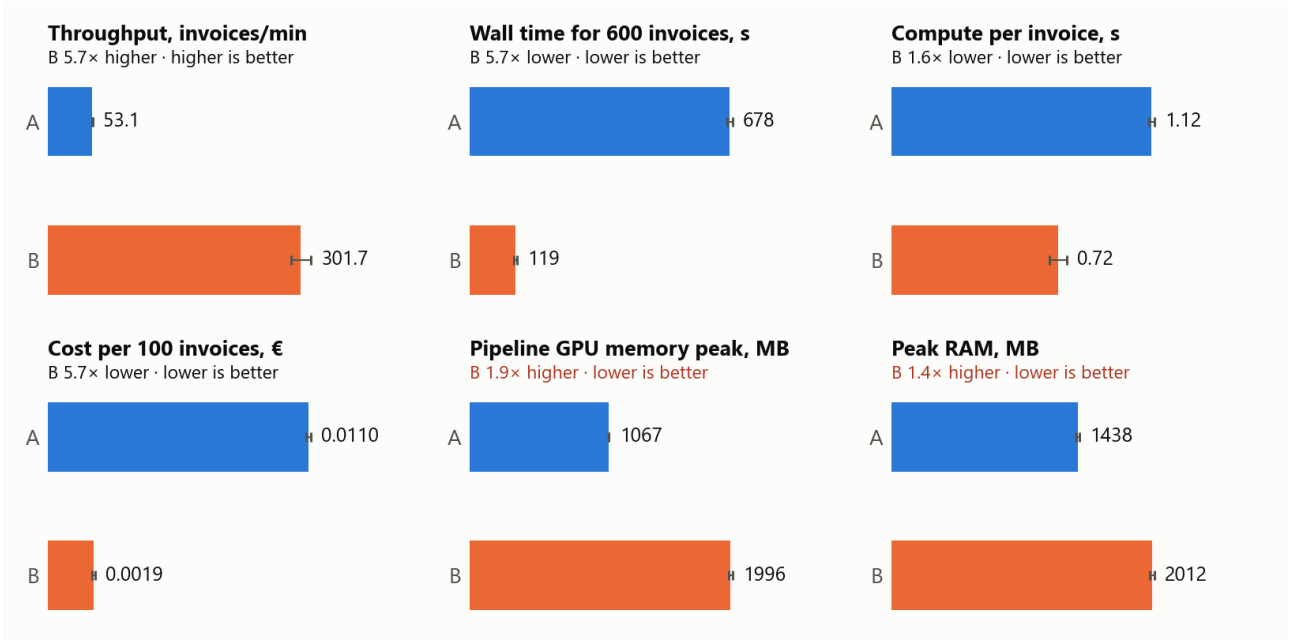


Figure 4 – A and B on the six performance measures (mean of 3 runs; the whiskers show the standard deviation). Red subtitles mark the measures on which B is worse.

Measure	Architecture A	Architecture B	B compared with A
Wall time for 600 invoices	677.9 ± 7.4 s	119.4 ± 4.7 s	5.7× faster
Throughput	53.1 ± 0.6 /min	301.7 ± 11.8 /min	5.7× higher
Service time per invoice	1.119 ± 0.012 s	0.718 ± 0.037 s	1.6× lower
of which OCR	0.588 s	0.576 s	same
of which line filter	0.017 s	0.008 s	2.0× lower
of which LLM	0.514 s	0.134 s	3.8× lower
p95 latency (all 600 submitted at once)	1.79 ± 0.03 s	112.7 ± 4.4 s	see 6.2
Pipeline GPU memory peak	1067 MB	1996 MB	1.9× higher

Measure	Architecture A	Architecture B	B compared with A
Whole-GPU memory peak (incl. other programs)	1388 MB	3172 MB	fits in 4 GB
Peak RAM, all processes	1438 ± 14 MB	2012 ± 16 MB	1.4× higher
Cost per 100 invoices	€0.0110	€0.0019	5.7× lower
F1, all 600 invoices	0.9406	0.9406	equal

6.2 Throughput over time and latency

B's advantage is throughput on a stream of invoices, not the speed of a single invoice.

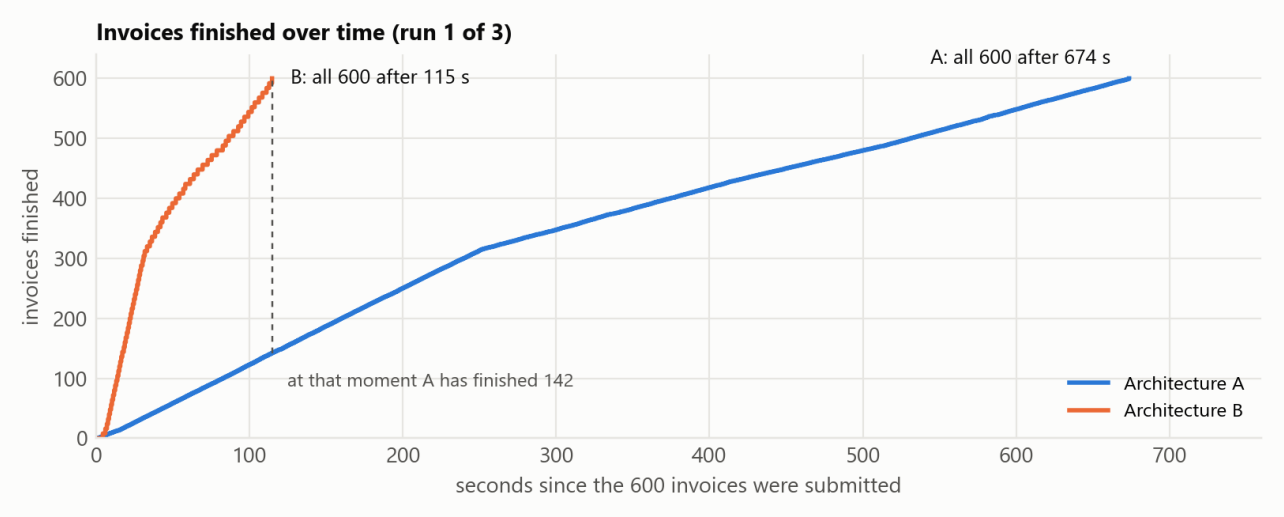


Figure 5 – Invoices finished over time in run 1. A's curve is built from its per-invoice times; B's from each invoice's time from submission to result. The bend in both curves is where the real invoices start, which take longer to read than the synthetic ones.

- In the time B needs for all 600 invoices, A finishes about a quarter of them.
- B's first result arrives after 4.4 s, A's after 0.6 s: B waits until a batch of 8 has been read, then answers all 8 at once.
- On a single invoice there is nothing to overlap or batch, so both take about the same time (1.32 s for A, 1.31 s for B on invoice 318).

6.3 Where the time goes per invoice

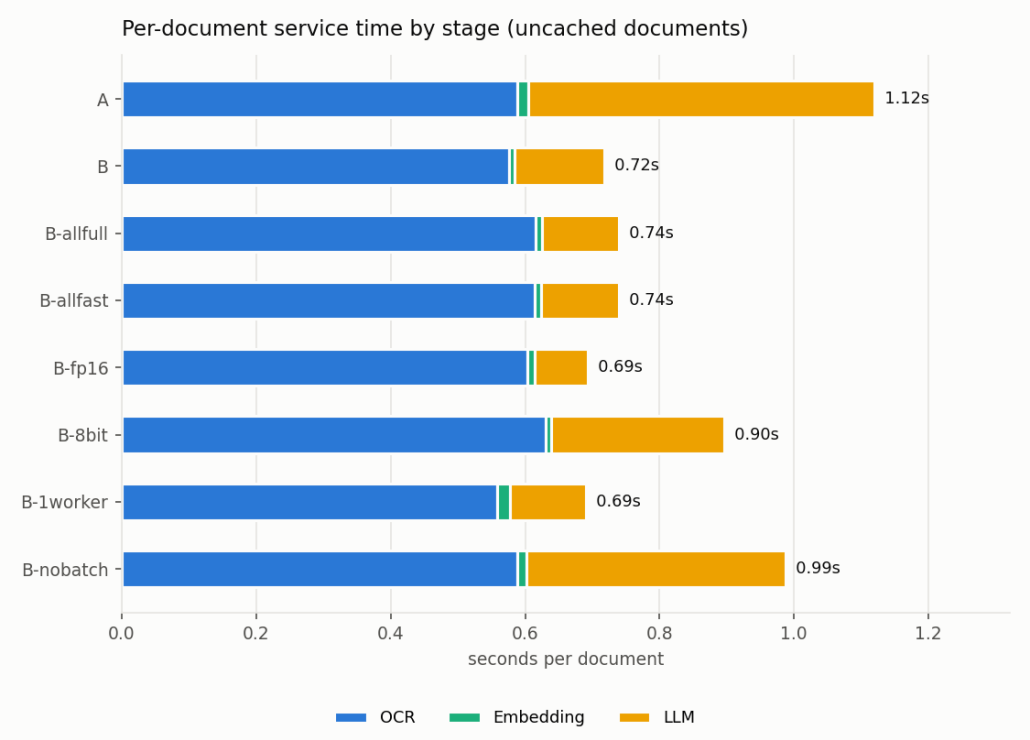


Figure 6 – Service time per invoice, split by stage. OCR costs the same in every configuration; B saves in the LLM stage through batching. The rest of B's speed-up comes from running stages at the same time, which a per-invoice view cannot show.

6.4 Where B's speed comes from

Each row changes one part of B; everything else stays the same.

Configuration	Wall time for 600	Throughput	What it shows
B as designed (3 OCR workers, batch 8)	119.4 ± 4.7 s	302 /min	reference
B-1worker: 1 OCR worker	338.4 ± 5.1 s	106 /min	parallel OCR is worth 2.8×
B-nobatch: batch size 1	248.8 ± 2.5 s	145 /min	GPU batching is worth 2.1×
B-6workers: 6 OCR workers	102.9 ± 10.3 s	352 /min	16 % faster again, but 2.6 GB RAM and the largest run-to-run spread; the GPU becomes the slower stage
B-nocache: no cache	125.9 ± 2.6 s	286 /min	the cache lookup costs nothing measurable on a first pass

A simple model explains all of these numbers. The OCR pool needs about 600 × OCR time ÷ workers; the GPU thread needs about 600 × (line filter + LLM time). Because the two stages overlap, B's wall time is close to the larger of the two.

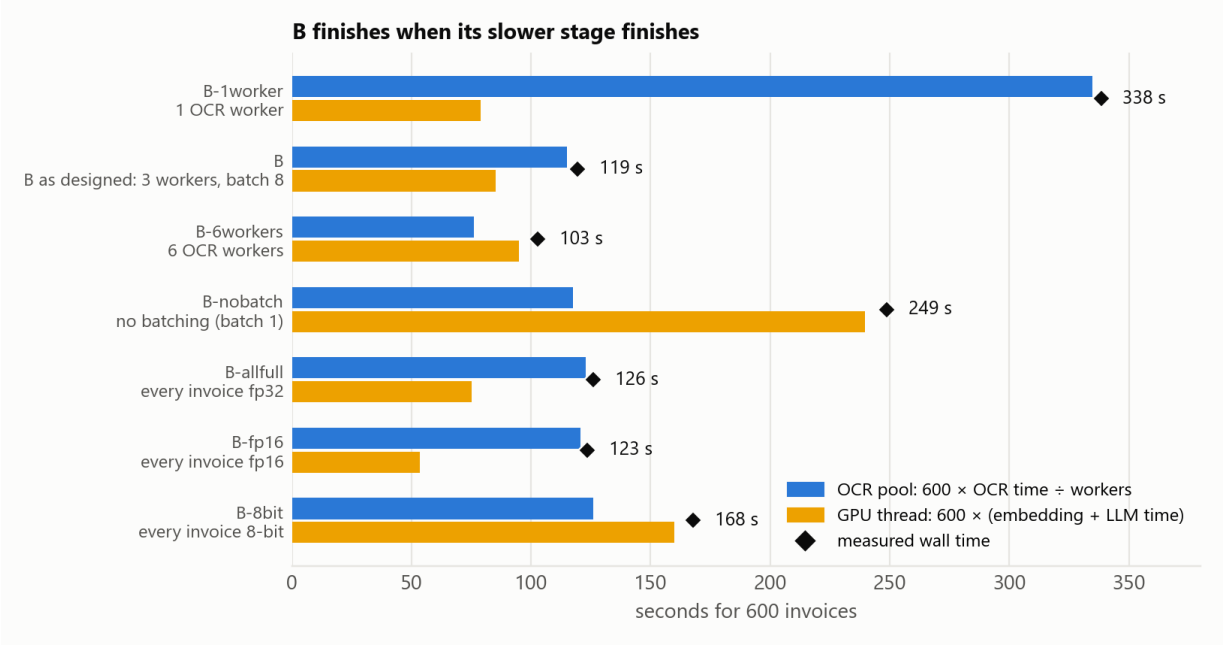


Figure 7 – Time each stage of B needs for 600 invoices (from the mean stage times) and the measured wall time. With 3 workers OCR is the slower stage, so a faster model barely changes the total; without batching or with 8-bit, the GPU is the slower stage.

6.5 Memory

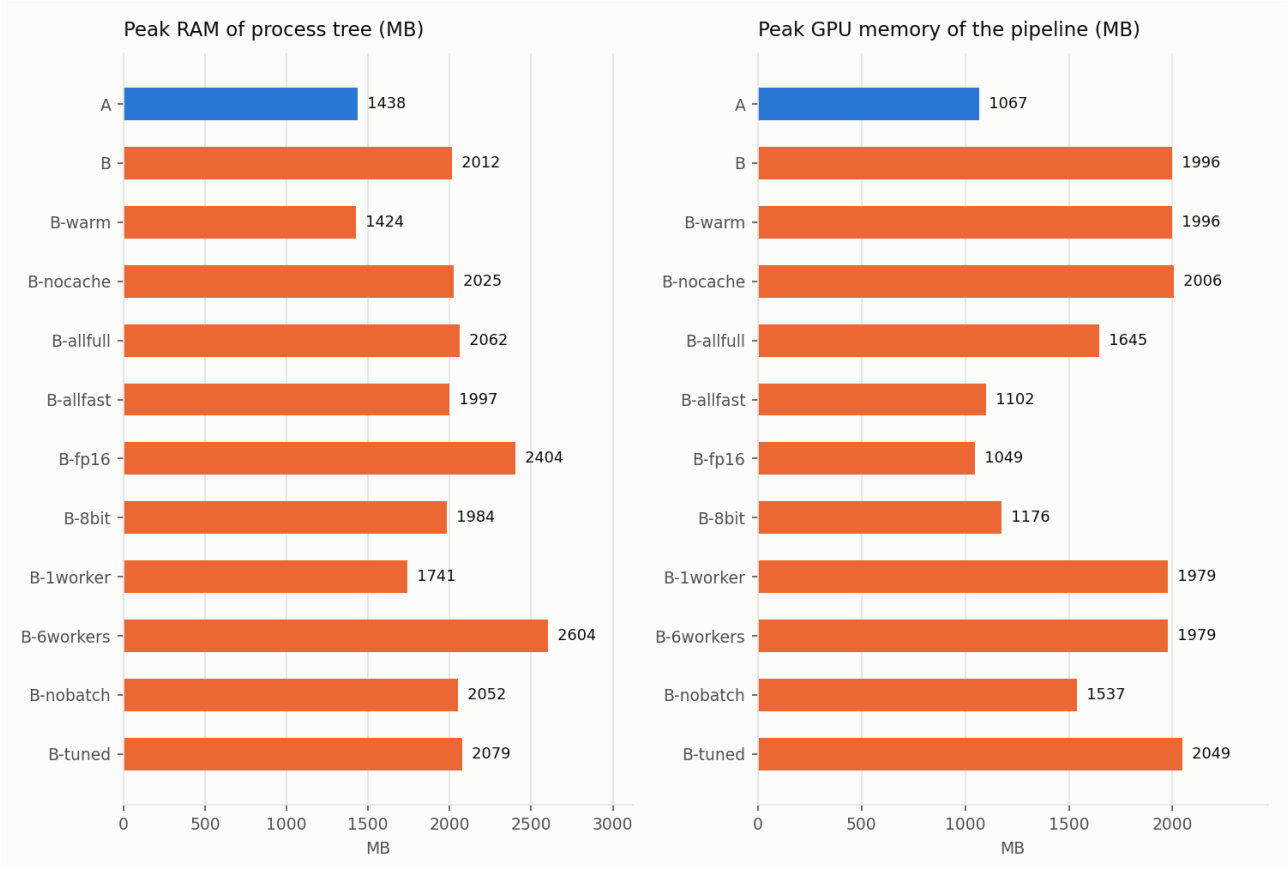


Figure 8 – Peak RAM of the whole process tree (left) and PyTorch's peak GPU memory (right), mean of 3 runs.

- B uses more GPU memory because it keeps the fp32 and the 4-bit model loaded together and processes 24 prompts at once. Its whole-GPU peak of 3172 MB still fits the 4 GB card.
- B uses more RAM because of its three OCR worker processes; with six workers RAM rises to 2604 MB.
- When only one model is needed, memory drops: 1102 MB with only the 4-bit model, 1049 MB with fp16.

6.6 Accuracy

A and B are equally accurate, by design: they share every step that produces an answer.

Code version	Arch.	F1 all 600	F1 real (300)	F1 synthetic (300)	Real: number / date / total	Real invoices fully correct
Before the accuracy upgrade	A	0.847	0.704	0.989	0.93 / 0.86 / 0.32	24 %
	B	0.850	0.712	0.987	0.93 / 0.86 / 0.34	26 %
Current code: row text + total check	A	0.941	0.892	0.989	0.94 / 0.92 / 0.82	72 %
	B	0.941	0.894	0.987	0.94 / 0.92 / 0.83	72 %

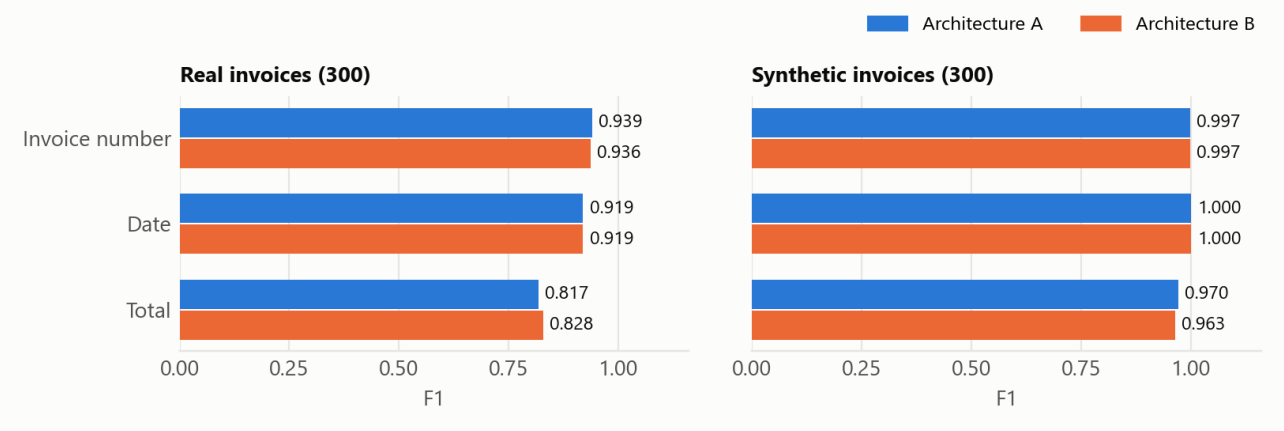


Figure 9 – F1 per field for real and synthetic invoices (run 1; accuracy does not vary between runs of A or B).

- The small per-field differences between A and B come from B's FAST route (4-bit model): on the FULL route the two give identical answers.
- The total on real invoices is the hardest field (F1 0.82–0.83); synthetic invoices are near 1.0 on every field.
- The accuracy upgrade (row-by-row OCR text and the total check) raised F1 from 0.85 to 0.94 in both architectures; it changed the shared components only.

6.7 Human review

Invoices with a review flag go to a person; the rest are accepted automatically.

	Architecture A	Architecture B
Accepted without review	92.0 %	92.5 %
Fully correct among accepted invoices	90.9 %	90.5 %
Flagged invoices that really were wrong	94 % (45 of 48)	93 % (42 of 45)
Wrong invoices caught by the flag	47 % (45 of 95)	44 % (42 of 95)
Real invoices: accepted / fully correct among accepted	87.3 % / 81.3 %	89.0 % / 80.5 %

Almost every flagged invoice really needed checking, but the flag catches fewer than half of the wrong invoices: a wrong invoice number or date that looks valid passes every check. Those are the main remaining errors.

6.8 Model precision and quantization

The same B pipeline with every invoice on one model, loaded in four precisions:

Precision	Wall time for 600	LLM time per invoice	Pipeline GPU peak	F1
fp32	126.0 ± 2.6 s	0.115 s	1645 MB	0.9406
fp16	123.5 ± 0.1 s	0.079 s	1049 MB	0.9406
4-bit NF4	125.8 ± 0.8 s	0.115 s	1102 MB	0.9406
8-bit LLM.int8	167.8 ± 1.6 s	0.258 s	1176 MB	0.9385 ± 0.0003

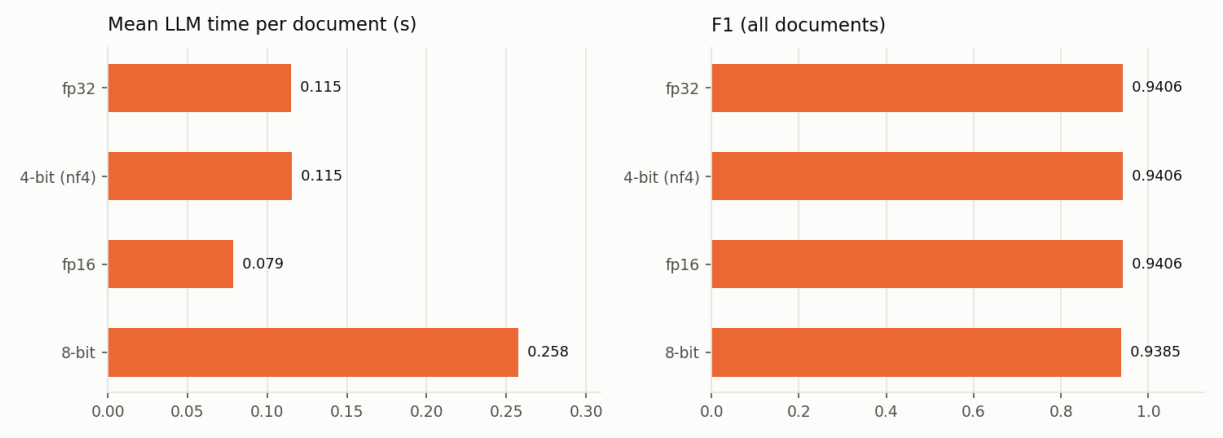


Figure 10 – LLM time per invoice (left) and F1 (right) for each precision.

- **4-bit saves memory, not time:** 33 % less GPU memory than fp32 at the same speed and accuracy.
- **fp16 is the best option on this GPU:** the fastest LLM step, the least memory and the same accuracy. For a 248 M-parameter model, quantization brings no advantage over half precision.
- **8-bit is the slowest** (2.2× the LLM time of fp32), and its outlier handling makes results vary slightly between runs.
- Because B is OCR-bound, a faster LLM step shortens B's wall time only a little (section 6.4).

6.9 Routing

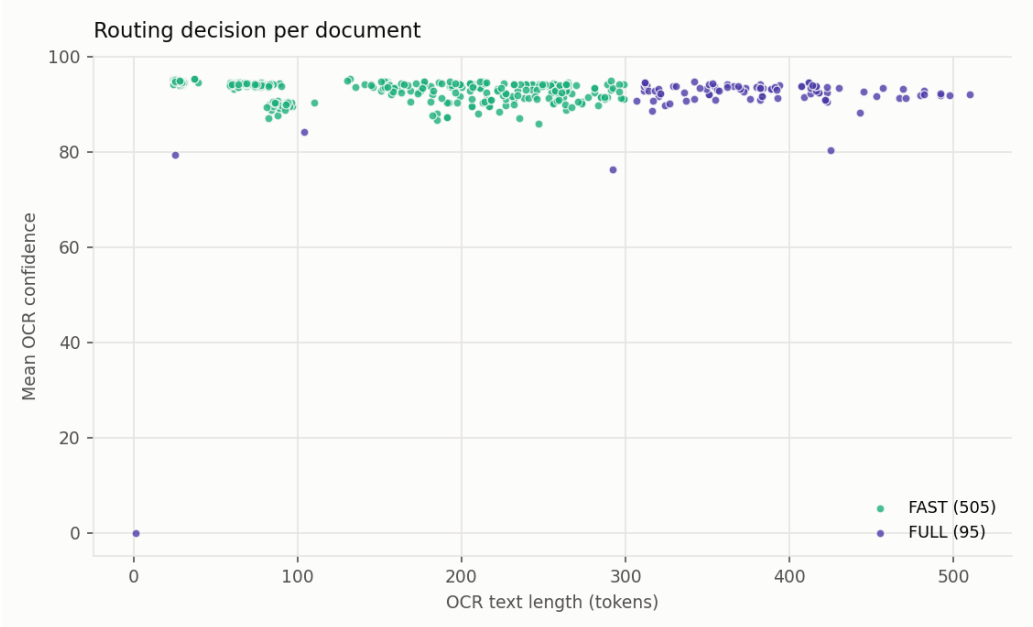


Figure 11 – Routing decision per invoice in B with the initial thresholds (300 tokens, OCR confidence 85).

- With the initial thresholds B sends 505 invoices to the 4-bit model and 95 to fp32.

- **Thresholds derived from the data:** a grid search over the all-4-bit and all-fp32 runs keeps field accuracy within 0.5 percentage points of all-fp32 and picks 525 tokens and confidence 70, which sends 99.8 % of invoices to the 4-bit model. The two models disagree on only 10 invoices, 5 each way. Three fresh runs (B-tuned) confirm it: F1 0.941, 126.1 s.
- The router works, but with two precisions of the same small model it has little to choose between. It pays off when the routes differ more, for example a small model for simple invoices and a larger one for complex ones.

6.10 Cache

Run	Wall time for 600	Cache hits	Per invoice
B, first pass (cache empty)	119.4 ± 4.7 s	0	199 ms of wall time
B-warm, same invoices again	0.59 ± 0.03 s	600	≈ 1 ms (hash + lookup)

6.11 Latency with invoices arriving one at a time

With invoices arriving one at a time, B as benchmarked is slower than A at light load – median 7.9 s against 1.7 s at 30 invoices per minute – because it waits until 8 invoices are read. Dynamic batching removes the wait: its median latency stays between 0.95 and 1.8 s at every load, the lowest of the three.

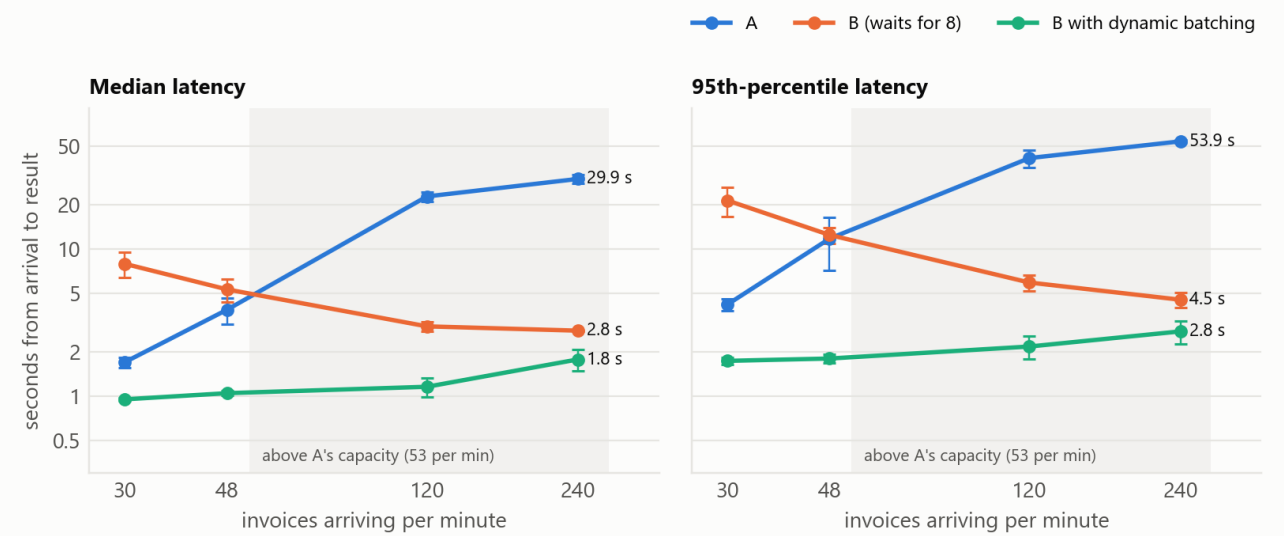


Figure 12 – Latency from arrival to result for 60 invoices arriving at random (Poisson) times, mean of 3 runs (whiskers: standard deviation, log scales). The grey area is above A's capacity, where A's queue keeps growing.

Arrival rate	A median	A p95	B median	B p95	Dynamic median	Dynamic p95
30 /min	1.7 ± 0.1	4.2 ± 0.4	7.9 ± 1.6	21.4 ± 4.8	1.0 ± 0.0	1.7 ± 0.1
48 /min	3.9 ± 0.8	11.8 ± 4.6	5.3 ± 0.9	12.5 ± 1.5	1.0 ± 0.0	1.8 ± 0.1
120 /min	22.8 ± 1.6	41.5 ± 5.6	3.0 ± 0.2	5.9 ± 0.7	1.2 ± 0.2	2.2 ± 0.4
240 /min	29.9 ± 2.0	53.9 ± 1.1	2.8 ± 0.0	4.5 ± 0.5	1.8 ± 0.3	2.8 ± 0.5

Seconds from arrival to result, mean ± standard deviation over 3 runs. Dynamic batching = B where the GPU, whenever it is free, takes every invoice already read (up to 8) instead of waiting for 8. Data: `outputs/2026-10-20_01-01-33_latency_60docs`.

- **A** answers quickly while it keeps up, but it handles only about 53 invoices per minute. Above that its queue grows with every invoice: median 22.8 s at 120 per minute and 29.9 s at 240 per minute after only 60 invoices – a longer stream would wait longer still.
- **B as benchmarked** gets faster as load rises (7.9 s → 2.8 s), because its batches of 8 fill sooner. At light load its batching costs more time than it saves.

- **Dynamic batching** keeps the throughput advantage and removes the wait. It spends a little more GPU time per invoice at light load (service time 0.98 s against 0.77 s at 30 per minute, because batches are small), which costs nothing while OCR is the bottleneck.
- Accuracy is the same in all 36 runs (F1 0.9444 on these 60 invoices): batching changes when an invoice is processed, not what is extracted.

6.12 OCR engines: Tesseract compared with EasyOCR

Tesseract is slightly more accurate than EasyOCR on this data (F1 0.941 vs 0.934), 9× faster on real scans, and needs no GPU memory. Both engines read all 600 invoices and feed the identical Architecture A downstream, so the difference is the engine's.

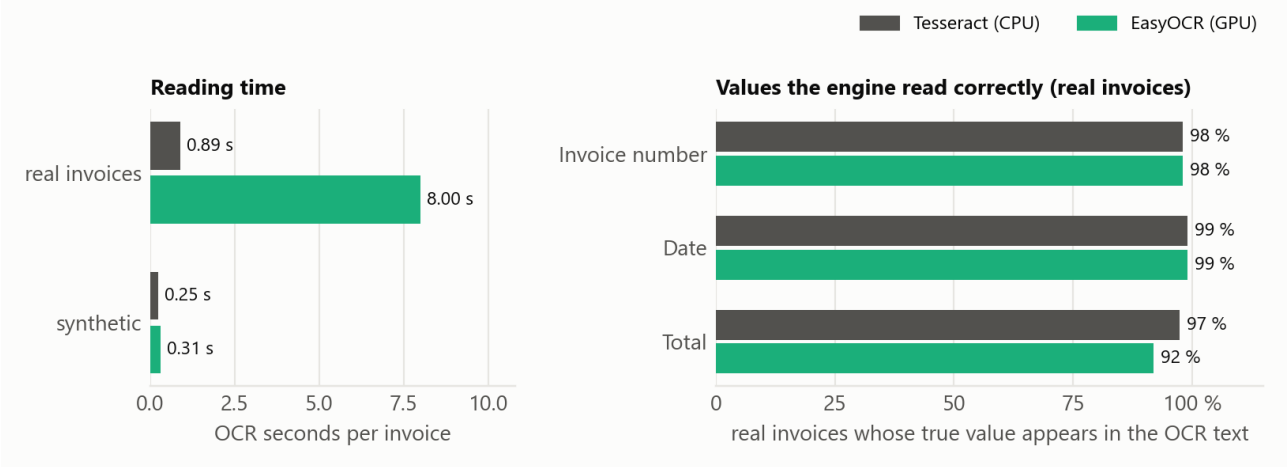


Figure 13 – Left: OCR time per invoice. Right: share of real invoices whose ground-truth value appears anywhere in the OCR text – an upper limit for any extractor working on that text.

Engine	Runs on	OCR time per invoice (mean / p95)	GPU memory for OCR	F1 all	F1 real	F1 synthetic	Fully correct
Tesseract 5.4	CPU, 1 thread	0.57 / 1.22 s	none	0.941	0.892	0.989	84.2 %
EasyOCR 1.7.2	GPU	4.16 / 13.49 s	3883 MB	0.934	0.871	0.997	82.7 %

Data: `outputs/2026-10-20_01-52-39_ocr_comparison`. One pass over 600 invoices per engine; the model is loaded before timing starts.

- **Speed:** on the real scans EasyOCR needs 8.0 s per invoice against 0.89 s for Tesseract (9× slower), although it runs on the GPU; the large 2550 × 3301 scans take about 14 s each. On the small synthetic images both take about 0.3 s.
- **Memory:** EasyOCR's peak of 3883 MB is nearly the whole 4 GB card, so it cannot share the GPU with the language model; Tesseract uses no GPU memory, which is what lets B overlap OCR on the CPU with extraction on the GPU.
- **Accuracy:** F1 0.941 (Tesseract) against 0.934 (EasyOCR). The gap is on real invoices (0.892 vs 0.871) and mostly in the total: the true total appears in Tesseract's text for 97 % of real invoices, in EasyOCR's for 92 %. On the clean synthetic invoices EasyOCR is slightly better (0.997 vs 0.989).
- **Scoring correction found here:** EasyOCR writes dates such as 'April 19,2024' without a space after the comma, which the date parser read with the current year. The parser was fixed (pipeline/normalize.py, with a unit test) and every stored run was re-scored. The main benchmark did not change – its labels and predictions shared the same format – while EasyOCR's F1 rose from 0.852 to its value above.

7. Is OCR still needed? A vision-language model on the same GPU

On the RTX 3050 a 2-billion-parameter VLM reads the invoices more accurately than the OCR pipeline (F1 0.966 against 0.941), but it needs 20× B's time per invoice (3.9 s against 0.20 s of wall time) and 3.5× A's. It runs on the 4 GB card in 4-bit (its 16-bit weights alone take 4.4 GB) with the image capped at 1024 visual tokens, using 1745 MB of GPU memory (A 1067 MB, B 1996 MB). On this hardware the case for OCR therefore rests on throughput, latency and cost, not on accuracy.

System	Reads	Wall time per invoice	Invoices / min	Pipeline GPU peak	F1 all	F1 real	Real: number / date / total
A	OCR text	1.13 s	53	1067 MB	0.941	0.892	0.94 / 0.92 / 0.82
B	OCR text	0.20 s	302	1996 MB	0.941	0.894	0.94 / 0.92 / 0.83
Qwen2-VL-2B, 4-bit, ≤ 1024 visual tokens	image	3.90 s	15.4	1745 MB	0.966	0.930	0.99 / 0.96 / 0.85

A and B: 600 invoices, mean of 3 runs. VLM: one pass, one invoice at a time, same normalisation and scorer. Data: `outputs/2026-10-20_23-54-32_vlm_baseline`.

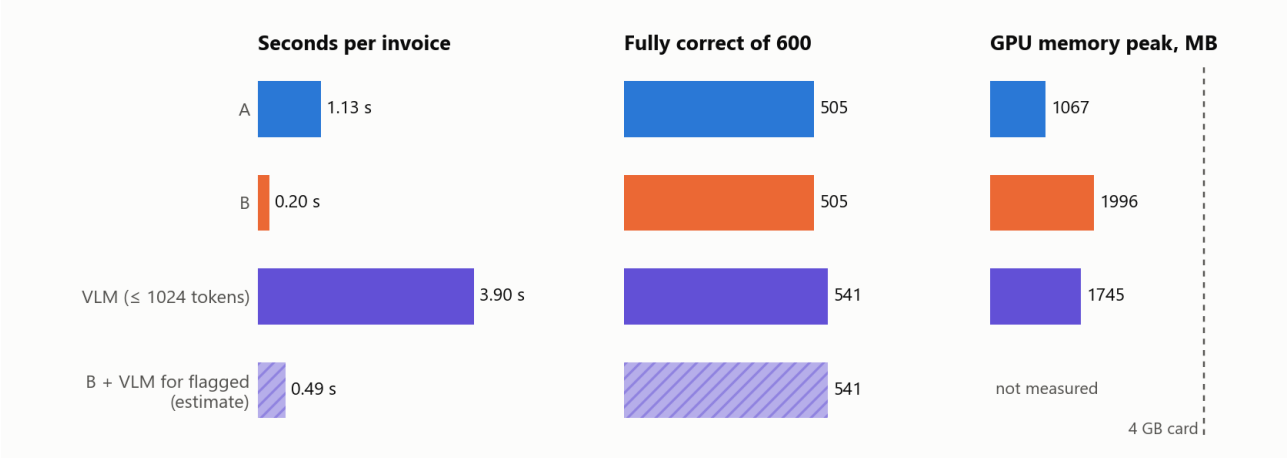


Figure 14 – Wall time per invoice, invoices with all three fields correct, and peak GPU memory. A and B: mean of 3 runs; VLM: one pass over the 600 invoices. The hatched row is an estimate from stored results: B's answers, with the VLM's answers for the invoices B's review flag sends to a person.

- **Memory decides what is possible.** The model's 16-bit weights alone (4.4 GB) exceed the card, so it runs in 4-bit NF4. Image resolution then trades accuracy against time and memory; a 20-invoice probe, one setting per process, gave ≤ 512 tokens: F1 0.95, 3.2 s per invoice, 1568 MB; ≤ 1024 tokens: F1 1.00, 3.8 s per invoice, 1745 MB; ≤ 2048 tokens: F1 0.95, 5.2 s per invoice, 2329 MB. (`outputs/2026-10-21_00-35-48_vlm_resolution_probe_20docs`)
- **Time.** 3.9 s per invoice one at a time (15 invoices per minute), against 1.12 s for A and 0.20 s of wall time per invoice for B. All of the VLM's work runs on the GPU, so there is no CPU stage to run in parallel as OCR is in B.
- **Accuracy by field (real invoices).** VLM: invoice number 0.99, date 0.96, total 0.85; B: 0.94, 0.92, 0.83 (same order). Valid JSON in 100 % of answers.
- **Using both.** On the 45 invoices that B's review flag sends to a person, B is fully correct 3 times and the VLM 39 times. Sending only those to the VLM would raise fully correct invoices from 505 to 541 of 600 at about 0.29 s of extra GPU time per invoice on average (estimated from the stored results).
- **In the literature.** The source list (Literature_Sources.pdf, section 2) has both sides: studies where OCR text improves even large VLMs or where a text model on OCR text beats a VLM of the same size, measured VLM memory far above 4 GB at default settings, and counter-evidence that large cloud VLMs need no OCR. One small model with one prompt was measured here; larger VLMs were not tested.

8. Improvement options

Options ordered by how well this project's data supports them. The right-hand column says whether the effect was measured, estimated from stored results, or is a design option without a measurement.

Option	Evidence	Effect	Basis
Dynamic batching in B: when the GPU is free, take every invoice already read (up to 8) instead of waiting for 8	B waits for a full batch; at light load an invoice waits for the next seven to arrive	Median latency at 30 invoices/min: 7.9 s → 1.0 s; at 240/min: 2.8 s → 1.8 s	measured (6.11)
Speed up OCR, B's bottleneck: more CPU cores or a faster CPU OCR engine	With 3 workers OCR needs about 115 s for 600 invoices, the GPU about 85 s	B's wall time follows the OCR pool's time (Figure 7). EasyOCR is not the faster engine here: 4.16 s instead of 0.57 s per invoice, with 3883 MB of GPU memory; a faster CPU OCR engine or more cores would help	measured (6.4, 6.12)
fp16 instead of 4-bit on the FAST route	fp16: fastest LLM step and least memory at the same F1 on this GPU	LLM time 0.115 → 0.079 s per invoice; small effect on the total while OCR is the bottleneck	measured (6.8)
Label checks: the invoice number must appear on a line with an invoice-number label (Invoice, No., #), the date on a line with a date label	The current review flag catches 42 of the 95 wrong invoices; most remaining errors are a wrong but real value copied from elsewhere on the page	Catches 64 of 95; fully correct among auto-accepted invoices 90.5 % → 94.0 %; straight-through rate 92.5 % → 86.2 % (more invoices go to a person)	estimated from stored B predictions and Tesseract text
Send only the flagged invoices to a VLM	45 of 600 invoices are flagged; on them B is fully correct 3 times, the VLM 39 times	Fully correct invoices 505 → 541 of 600 (the VLM alone on every invoice: 541), for about 0.29 s of extra GPU time per invoice – about 1/13 of running the VLM on everything	estimated from stored B and VLM results
Route between genuinely different models, not two precisions of one model	The 4-bit and fp32 copies of Flan-T5 disagree on only 10 of 600 invoices, so the router has little to choose between	Routing pays off when the routes differ in cost and accuracy, e.g. Flan-T5 for clean invoices and a larger model or a VLM for hard ones	measured (6.9), effect not measured
Shared cache and queue for several machines	SQLite and an in-process event loop serve one workstation	Redis (shared cache) and a message queue would let several machines share work and results	design option, not measured

9. Limits of these measurements

- One machine: a laptop with a 4 GB RTX 3050 and 12 CPU threads. The balance between OCR and GPU – and so which stage limits B – will differ on other hardware.
- Half of the dataset is synthetic and easy; the real half (300 invoices) carries the accuracy differences.
- Cost is a model: wall time × €0.35 per GPU-hour. It ignores CPU price, idle time and model loading.
- The main benchmark submits all 600 invoices at once. Latency with invoices arriving one at a time was measured separately on 60 invoices at 4 arrival rates (section 6.11).
- The OCR comparison and the VLM baseline are single passes over the 600 invoices; their accuracy is deterministic, their timings are means over 600 invoices rather than over 3 runs.
- The VLM baseline tests one model (Qwen2-VL-2B, 4-bit) with one prompt; other small VLMs or prompts may score differently, and larger VLMs were not tested (MiniCPM-V 8B, for example, is documented to need about 5 GB even in 4-bit – Literature_Sources.pdf, section 2.3).
- B-6workers varies most between runs (± 10.3 s), as the laptop's CPU threads saturate.
- Ground truth: 26 totals on real invoices were corrected from net to gross, each checked by net + VAT = gross on the OCR text; the originals are kept in `ground_truth_backup_20261018/`.

Appendix: reproducing the benchmark

Open PowerShell in `G:\Freelance\ocr_llm_project\ocr_llm_project`.

What	Command
Unit tests	`.venv\Scripts\python -m pytest tests -q`
A and B side by side on 30 invoices	`.venv\Scripts\python demo.py --docs 30`
Full benchmark, A and B only	`.venv\Scripts\python -m evaluation.run_experiments --configs A B --runs 3`
Every configuration (as in this report)	`.venv\Scripts\python -m evaluation.run_experiments --configs all --runs 3 --skip-existing`
Tables and figures	`.venv\Scripts\python -m evaluation.report`
Routing thresholds from data	`.venv\Scripts\python -m evaluation.routing_analysis`
Latency with invoices arriving one at a time	`.venv\Scripts\python -m evaluation.latency_experiment --docs 60 --runs 3`
Tesseract vs EasyOCR	`.venv\Scripts\python -m evaluation.ocr_comparison`
VLM baseline	`.venv\Scripts\python -m evaluation.vlm_baseline`

- Each benchmark writes a new folder `outputs/<date>_<time>_<label>/` with `summary.md` (all tables), `figures/`, `run_info.json` (command, settings, hardware, versions) and per run `runs/<configuration>/run_<k>/documents.csv` (one row per invoice).
- Keep the laptop plugged in and idle during a run. If OCR time per invoice is well above 0.7 s, something else used the CPU – repeat that run.
- A full benchmark of every configuration takes about 4 hours; `--skip-existing` resumes an interrupted one.